

PQC Key Agreement

Eric Grosse

April 21, 2026

Suppose Alice and Bob have an end-to-end encrypted communications system based on a pre-shared symmetric secret S . Now the time has come that they wish to rotate this secret.

They might use couriers or meet in person as we imagine they did years ago when first setting up their system. But if we're lucky they already in the past created static ML-KEM-768 keypairs and exchanged the public parts ek_A and ek_B . Using any available untrusted channel, they can perform key rotation as described in Figure 11 of NIST SP-800-227 section 5.2.5:

Alice generates ephemeral $ek, dk = \text{Keygen}()$ and $K_B, c_B = \text{Encaps}(ek_B)$ and sends ek, c_B to Bob.

Bob computes $K, c = \text{Encaps}(ek)$, $K_A, c_A = \text{Encaps}(ek_A)$, $K'_B = \text{Decaps}(dk_B, c_B)$ and sends c, c_A to Alice.

Alice computes $K' = \text{Decaps}(dk, c)$ and $K'_A = \text{Decaps}(dk_A, c_A)$. The new shared secret is computed as $\text{SHA3}(K', K'_A, K_B)$ by Alice and $\text{SHA3}(K, K_A, K'_B)$ by Bob.

This is implemented with `github.com/n2vi/hotline/puck.go` as follows:

1. Sometime in advance, Alice and Bob each execute `hotline mlkem-gen > MLKEMpub` and publish their 1184-byte binary file somewhere. They also note the base-64 sha256 hash of the file for confirming correct delivery of the file to correspondents.

Alice executes `hotline mlkem-pub bob < MLKEMpub` to load Bob's public key and confirms the hash. Bob does likewise.

These public keys are not secret, don't need to be changed often, and can be used for as many correspondents as many times as desired. Done during peaceful times, the publishing and fetching can use legacy complex infrastructure such as email or Signal or github or whatever. Confirming the hash requires some relatively trusted channel and might be the weakest link of the scheme. It is left as an exercise for the reader whether certificate authorities improve trust here.

2. Alice executes `hotline mlkem-chal bob > MLKEMchal` and sends that 2272-byte binary file to Bob.

Again this can be over legacy infrastructure, though during an extended crisis perhaps the key rotation could happen using the existing hotline itself. Perhaps you fear that a large volume of traffic has put the chacha20 secret at cryptanalytic risk. Since we don't depend on confidentiality or integrity of the channel, using the hotline for the rekey would still be ok. On the other hand if you fear your puck itself was compromised, then it is game over. Switch to an entirely new system for sure. But even in the more limited case, I worry that using the hotline for the rekey is a bit like sawing off the limb you're standing on; anyway, we don't support it for now.

3. Bob executes `hotline mlkem-resp alice < MLKEMchal > MLKEMresp` and sends that 2176-byte binary file to Alice. He now has a new shared secret as `MLKEMss` in the json file `PrincipalsDB` under Peer "alice".
4. Finally, Alice executes `hotline mlkem-shared bob < MLKEMresp`. She has the same `MLKEMss` under Peer "bob".
5. If desired, Alice and Bob might now do something to check that everything worked consistently. Or if they're feeling brave, Alice executes `hotline rekey bob mlkemss` and Bob executes `hotline rekey alice mlkemss` where `mlkemss` is the string (not including quotes) each finds in `PrincipalsDB`.